Operációs rendszerek

3. Folyamatok kommunikációja

Felhasznált irodalom:

- **Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben**
- **Tanenbaum: Modern Operating Systems 2nd. Ed.**
- **Silberschatz, Galvin, Gagne: Operating System Concepts**

v4.0

Tartalom

- Bevezetés
- Szinkronizáció
- A kritikus szakasz megvalósítási módozatai
- Információcsere folyamatok között

3.1 Folyamatokból álló rendszerek

- Független folyamatok
  - egymás működését nem befolyásolják (aszinkron)
    - nincs időbeli függőség
    - egymással párhuzamosan is végrehajtódhatnak
- Függő folyamatok
  - logikailag független folyamatok
    - osztott erőforrás használat miatt függőség
  - logikailag függő folyamatok
    - közösen oldanak meg valamely feladatot
    - együttműködnek, kommunikálnak

Együttműködő folyamatok szükségessége

- Erőforrások megosztása
  - átlapolt működés, jobb kihasználtság
- Számítások felgyorsítása (több processzor)
  - számítások párhuzamosítása, végrehajtási sebesség nő
- Felhasználók kényelme
  - egy időben több feladat megoldása
- Modularitás
  - egy adott folyamat kisebb részekre való bontása
  - jobb áttekinthetőség

3.2 Szinkronizáció

Definíció: A szinkronizáció egy folyamat végrehajtásának olyan időbeli korlátozása, ahol a végrehajtás folytatása egy másik folyamat futásától függ.

Gyakori feladatok:

- Precedencia (előidejűség)
- Egyidejűség
- Kölcsönös kizárás (versenyhelyzet, kritikus szakasz)

Kapcsolódó fogalom:

- Holtpont (deadlock)

Precedencia

- Meghatározott sorrend biztosítása.
- Egy P_k folyamat S_k utasítása csak akkor mehet végbe ha a P_i folyamat S_i utasítása már befejeződött.
- pl:

  szakács-kávét főz ← kukta-cukrot vesz

  (különben kihűl a kávé)

Egyidejűség

- Két vagy több folyamat bizonyos utasításait (S_k; S_j) <u>egyszerre</u> kell elkezdeni.
- Két folyamat találkozása (randevú).
- Két folyamat bevárja egymást mielőtt további működését elkezdené.
- pl.

  szakács-kávét főz || kukta-habot ver

  (különben kihűl a kávé vagy összeesik a hab)

Kölcsönös kizárás (mutual exclusion)

- A résztvevő folyamatok utasításainak sorrendjére nincs korlátozás, de egy időben <u>csak egyik</u> futhat.
- pl.

  szakács-habot ver X kukta-habverőt mosogat

Versenyhelyzet

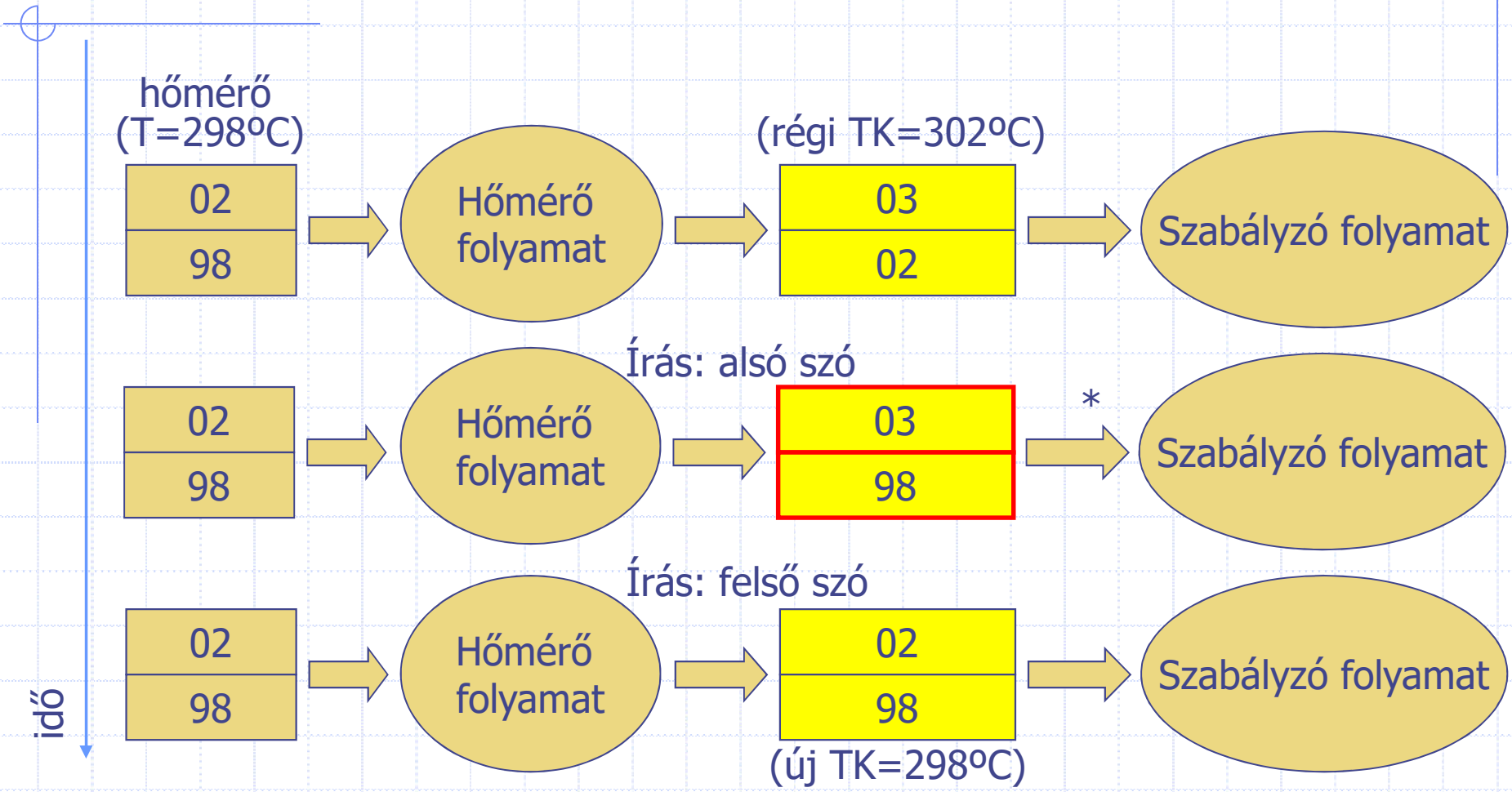
Definíció: Több párhuzamosan futó folyamat közös erőforrást használ. A futás eredménye függ attól, hogy az egyes folyamatok mikor és hogyan futnak, ezáltal milyen sorrendben férnek az erőforráshoz.

!!! Elkerülendő, nagyon nehéz debuggolni !!!

Példa:

A mért hőmérséklet (T) értékét egy két szó hosszúságú változóban (TK) tároljuk. A hőmérő folyamat a hőmérsékletet szavanként beírja a változóba, a szabályzó folyamat pedig kiolvassa a változót és annak értékét használja.

Példa versenyhelyzetre: hőmérséklet-szabályzó



*Ha ebben a pillanatban olvas, hibás, inkonzisztens értéket kap.

Kritikus szakasz

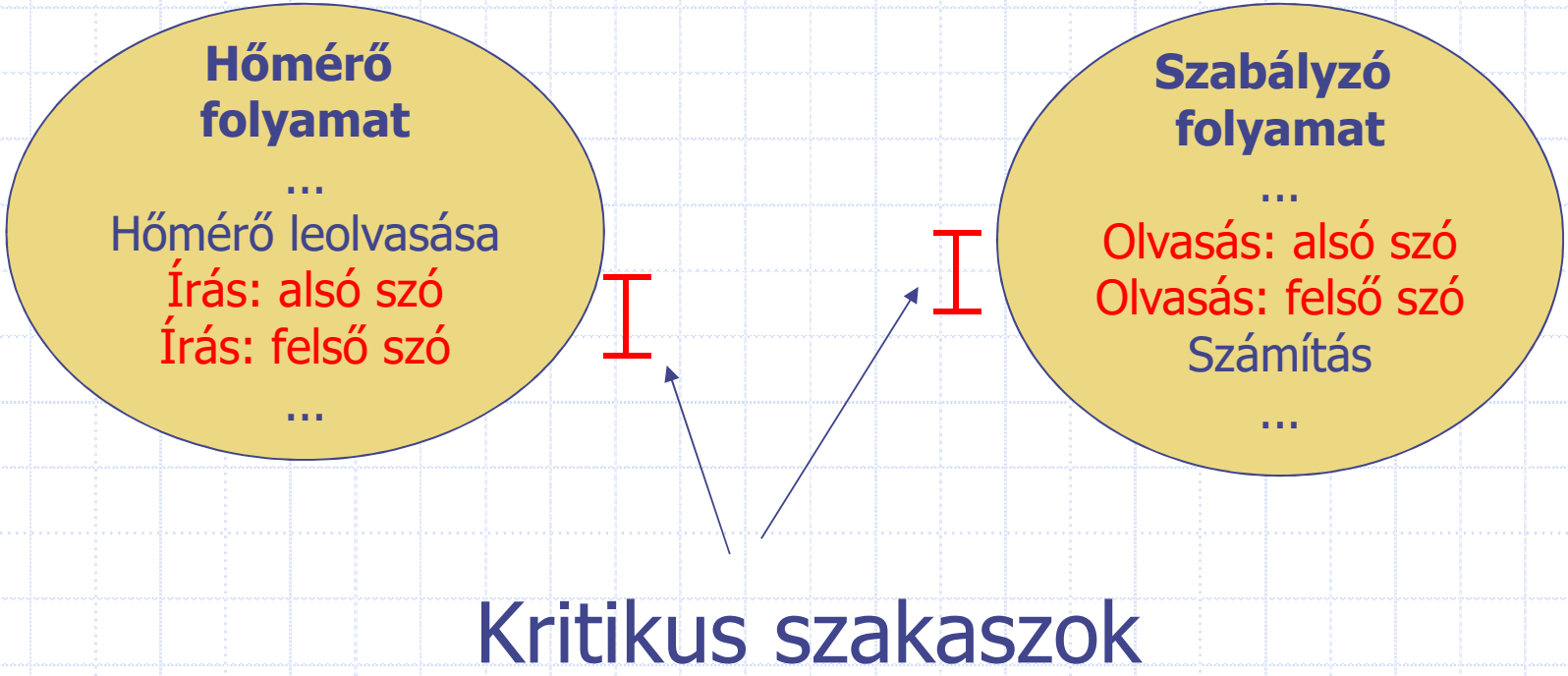
Definíció: Kritikus szakaszoknak nevezzük a program olyan utasításszekvenciáit, amelyeknek egyidejű végrehajtása nem megengedett.

- Versenyhelyzet elkerülésére a kritikus szakaszok kölcsönös kizárását biztosítani kell.

  (Ha az egyik folyamat már a kritikus szakaszában van, akkor más folyamat nem léphet be a (természetesen saját) kritikus szakaszába.)
- Pl.

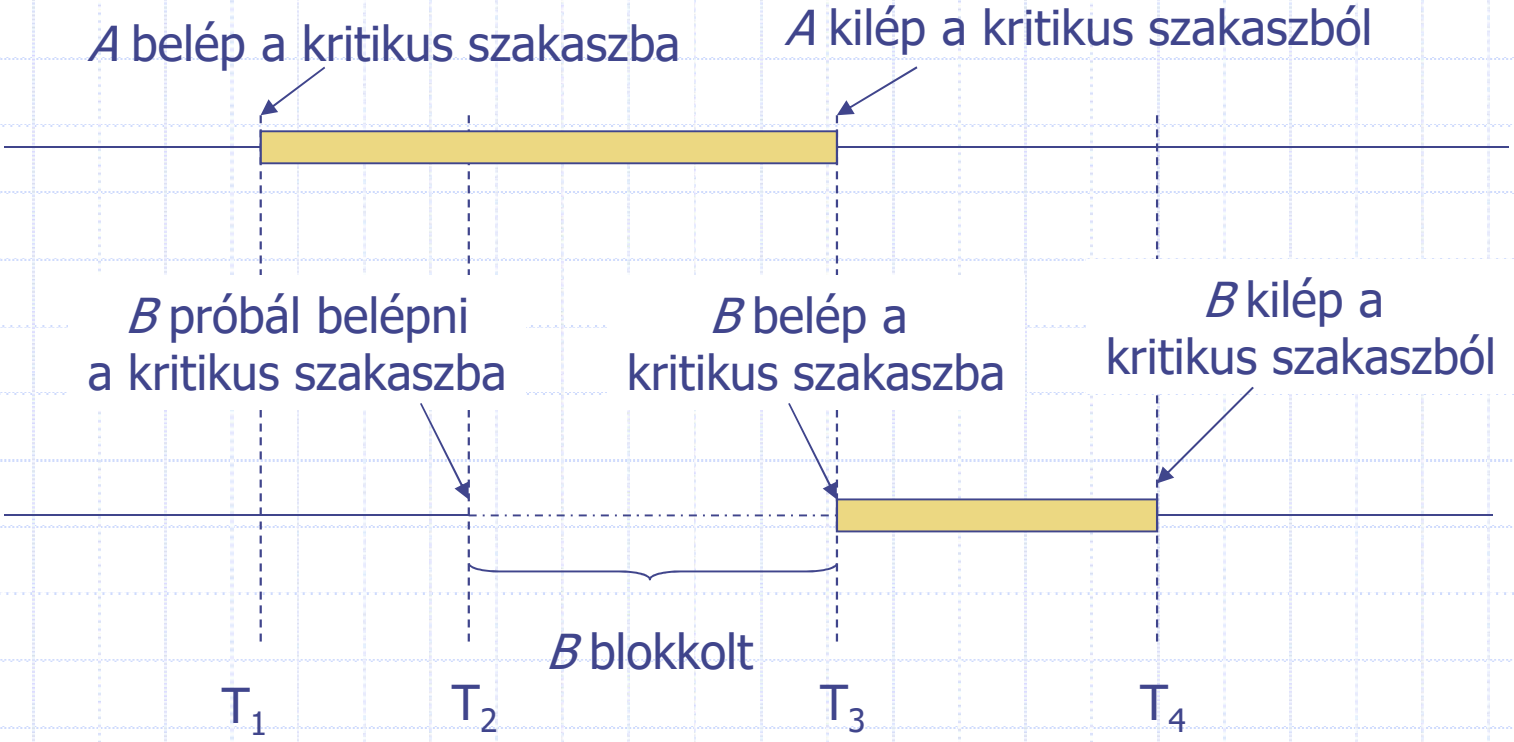
  Tipikus kritikus szakasz a közös memória használata.

Példa kritikus szakaszra: hőmérséklet-szabályzó



Kritikus szakaszok

Kritikus szakaszok és a kölcsönös kizárás



A kritikus szakaszban lévő *A* folyamat blokkolja *B* végrehajtását, amikor az is a kritikus szakaszba kíván lépni (T_2). Amikor *A* kilép a kritikus szakaszból (T_3), a blokkolás véget ér és *B* most már beléphet a kritikus szakaszba.

3.3 A kritikus szakasz megvalósítási kritériumai

A megvalósítás követelményei

1. Biztosítsa a kölcsönös kizárást
   - Egy időben csak egyetlen folyamat hajthatja végre a kritikus szakaszban lévő utasításokat.
2. Haladjon
   - Ha nincs folyamat a kritikus szakaszban, de van több belépő, akkor az algoritmus ezek közül véges idő alatt kiválaszt egyet és beengedi a kritikus szakaszba.
3. Folyamat várakozása korlátozott legyen (ne éheztessen)
   - Csak véges számú esetben előzhetik meg.

- A 3. kritériumot nem veszik annyira szigorúan (nem minden algoritmus teljesíti).

Kritikus szakasz megvalósítása

A folyamat általános struktúrája:

```
//Process i
while (TRUE){
    non_critical_region1();
    entry_section();
    critical_region(); //nem kell függvény legyen
    exit_section();
    non_critical_region2();
}
```

Kritikus szakasz megvalósítása

- Interrupt tiltása
- *Busy waiting* megoldások
  - Nincs HW támogatás, tiszta SW megoldás.
    - Naiv megközelítés (ROSSZ)
    - Strict alternation (JOBB)
    - Peterson algoritmusa (HELYES)
  - HW támogatással:
    - TestAndSet
- Szemafor
- Magas szintű módszerek

Interrupt tiltása

- Megoldás

```
disable_interrupt();
critical_region();
enable_interrupt();
```

- Egyszerű megoldás
  - Nincs INT, tehát nem lehet a végrehajtást megszakítani. (Ütemező sem tud futni!)
- Probléma
  - A folyamatnak joga van letiltani az INT-t. Az egész rendszert lefagyaszthatja.
- Hasznos az OS szintjén
- Nem célszerű a felhasználói szinten.

Naiv programozott megközelítés

- Közösen használt változókon alapul. (flag)
- Egy foglaltsági bitet használ, amit a belépni kívánó folyamat tesztel.

```
//Process 0
while (TRUE){
    while (flag!=0) /*loop*/;
    flag = 1;
    critical_region();
    flag = 0;
    non_critical_region();
}
```

```
//Process 1
while (TRUE){
    while (flag!=0) /*loop*/;
    flag = 1;
    critical_region();
    flag = 0;
    non_critical_region();
}
```

- Gond: a változót többen is olvashatják, mielőtt az első foglaltra állítja!

Szigorú váltás (*strict alternation*) algoritmus

- Közös változó: *turn*. Jelentése: ki van soron.
- Csak felváltva lehet belépni.

„Spin lock”

```
//Process 0
while (TRUE){
    while (turn!=0) /*loop*/;
    critical_region();
    turn = 1;
    non_critical_region();
}
```

```
//Process 1
while (TRUE){
    while (turn!=1) /*loop*/;
    critical_region();
    turn = 0;
    non_critical_region();
}
```

- Gond:
  - Ha jelentős a sebességkülönbség, akkor az egyik feleslegesen sokat vár.
  - Nem halad!

Peterson algoritmusa 1.

```
define FALSE 0
define TRUE 1
define N 2                          /* a folyamatok száma */
int turn;                           /* ki van soron? */
int interested[N];                  /* kezdetben csupa 0 */
void enter_region(int process);     /* a folyamat belép a kritikus szakaszba*/
{
    int other;
    other = 1 – process;            /* a másik folyamat */
    interested[process] = TRUE;     /* az érdeklődés jelzése */
    turn = process;                 /* flag beállítása */
    while (turn == process && interested[other] == TRUE) /* nop */;
}
void leave_region(int process);     /* a folyamat elhagyja a kritikus szakaszt */
{
    interested[process] = FALSE;
}
```

Peterson algoritmusa 2.

- Ha csak egy folyamat akar belépni, az `enter_region()` visszatér és a folyamat a kritikus szakaszba léphet.
- A másik folyamat addig nem léphet be, amíg a `leave_region()` az `interested` változót nem törli.
- Ha két folyamat egyszerre akar belépni, a turn változót utoljára állító folyamat várakozik, míg a másik végre nem hajtja a saját `leave_region()` eljárását.
- Megjegyzés: A `turn` változó írásának megszakíthatatlanak kell lenni (ez könnyen teljesíthető).

Hardver támogatás

Speciális **megszakíthatatlan** gépi utasítások

1. TestAndSet
   - Kiolvassa és visszaadja a bit értéket, majd azonnal 1-be állítja.

```
int TestAndSet(int *flag){
    tmp = *flag;
    *flag = 1;
    return(tmp)
}
```

oszthatatlan

2. Swap
   - Két változó értékét cseréli fel

```
void swap(int *a, int *b){
    tmp = *a;
    *a = *b;
    *b = tmp;
}
```

oszthatatlan

TestAndSet utasítás használata

```
//init
flag = 0;
...


//Process i
while (TRUE){
    while (TestAndSet(&flag)!=0) /*empty loop*/;
    critical_region();
    flag = 0;  /*exit critical section*/
    non_critical_region();
}
```

Nem garantálja az éhezés kiküszöbölését!

Swap utasítás használata

```
//init
flag = 0;
...

//Process i
...
while (TRUE){
    myflag = 1;
    while (myflag!=0) swap(&myflag, &flag);
    critical_region();
    flag = 0;  /*exit critical section*/
    non_critical_region();
}
```

Nem garantálja az éhezés kiküszöbölését!

Szemafor

Edsger Wybe Dijkstra, 1965

- Speciális adattípus (nemnegatív egészek)
- Két oszthatatlan utasítás

- **P(s)** - vizsgál/belép művelet

  Működési elv:

```
while s < 1 do üres utasítás;
s := s - 1
```

- **V(s)** – kilép művelet

  Működési elv:

```
s := s + 1
```

E.W. Dijkstra (1930-2002)

Holland kutató (számítástudomány)

Nevéhez fűződik

- Dijkstra algoritmus (legrövidebb út)
- Algol 60 nyelv (függvényhívás, rekurzió)
- Algol 60 compiler
- Konkurens programozás
- Szemafor
- Deadlock

Mutex (mutual exclusion)

- A mutex olyan speciális szemafor, amelynek csak két értéke lehet:
  - nyitott (unlocked)
  - zárt (locked)
- Műveletei:
  - mutex_lock
  - mutex_unlock

Szinkronizáció szemaforral I.

Előidejűség (U_1 előbb mint U_2)

```
s := 0
P1 : ... U1; V(s); ...
P2 : ... P(s); U2; ...
```

Randevú (U_1 és U_2 egyszerre)

```
s1 := 0; s2 := 0
P1 : ... V(s1); P(s2); U1; ...
P2 : ... V(s2); P(s1); U2; ...
```

Szinkronizáció szemaforral II.

Kölcsönös kizárás

```
s := 1
P1 : ... P(s); U1; V(s); ...
P2 : ... P(s); U2; V(s); ...
```

Az *s* kezdő értékétől függ, hogy hány folyamat lehet egyszerre a kritikus szakaszban. Legtöbbször bináris szemafort (mutex) használunk.

Szemaforok megvalósítása

Probléma a szemafor elvi megvalósításával: busy waiting

```
P(s):
while s < 1 do üres utasítás;
s := s - 1
```

- Ez blokkolt állapotban is állandóan futna, állandóan használná a CPU-t. Így praktikusan nem használható.

Multiprogramozott megoldás (OS támogatás):

- A *P*(*s*) az üres utasítás helyén a folyamattól elveszi a CPU-t és várakozó állapotba teszi (feljegyzi a szemaforhoz tartozó valamilyen adatszerkezetekbe).
- A *V*(*s*)-nél nem csak a számláló nő, hanem egy folyamatot (ha van) futásra kész állapotba tesz (többféle stratégia lehet).

Szemaforok megvalósítása

```
type semaphore  = record
                    value: integer
                    list : list of process
                  end; s
P(s):
   s:value := s:value - 1
   if s:value < 0 then
   begin
        a folyamat felfűzése a s.list-re
        a folyamat felfüggesztése, újra ütemezés
   end

V(s):
   s:value := s:value + 1
   if s:value <= 0 then
   begin
        leveszünk egy folyamatot s:list -ről
        felébresztjük ezt a folyamatot
   end
```

Monitor

- A szemafor hasznos szinkronizációs eszköz, de alacsony szintű.
  - Sok hibalehetőség, ráadásul nehéz a hibakeresés.
- Nyelvi elemekkel való támogatás: *monitor*, egy magas szintű szinkronizációs primitív.
- A monitor eljárások, változók, adatszerkezetek speciális gyűjteménye.
- A monitor eljárásai szabadon hívhatóak, de a változókhoz nem lehet kívülről közvetlenül hozzáférni.
- Egyszerre csak egy eljárás lehet aktív a monitoron belül!

Monitor példa

Egyszerű monitor példa:

- Csak a *producer* és a *consumer* eljárások láthatók
- Az *i* és *c* belső változókat csak a belső eljárások kezelhetik
- Legfeljebb egy eljárás lehet aktív: vagy a *producer*, vagy a *consumer*

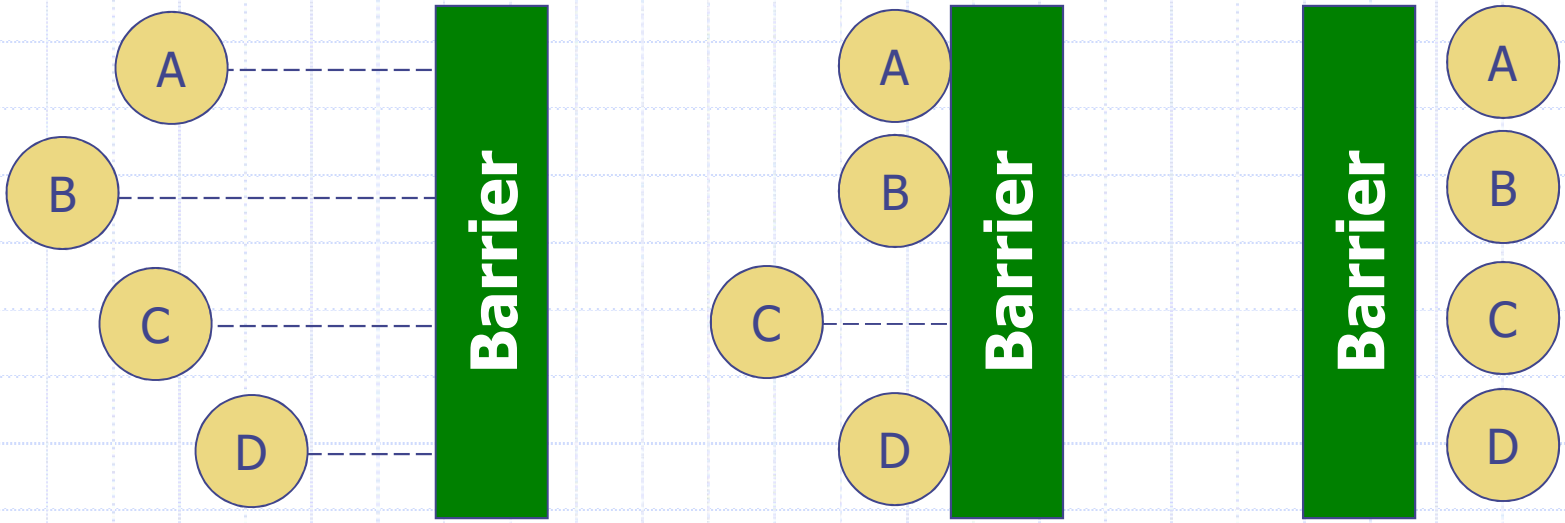
```
monitor example
        integer i;
        condition c;

        procedure producer();
        …
        end;

        procedure consumer();
        …
        end

end monitor;
```

Az akadály (barrier)



a) Több folyamat (A, B, C, D) szinkronizálására szolgál
b) Az akadályt elérő folyamatok blokkolódnak
c) Az összes folyamat megérkezésekor az akadály ledől, az összes folyamat egyszerre folytatja futását.

3.4 Információcsere folyamatok között

Együttműködéséhez szükséges.

1. Közös tárterületen keresztül
   - szorosan csatolt megoldás
   - kölcsönös kizárás

2. Kommunikációs csatornán keresztül
   - lazán csatolt megoldás
   - Két folyamat között valamilyen - fizikai, virtuális (logikai) - csatorna van.
   - Egyik folyamat küld, a másik vesz.

Kommunikációs csatorna tulajdonságai

- Átviteli közeg:
  - Pl: közös tár, lokális hálózat
- Kapcsolat iránya:
  - egy- vagy kétirányú
- Két vagy több folyamat használja
  - két folyamat között pont-pont kapcsolat
  - több folyamat esetén
    - időben multiplexelve
- Van-e közbülső tárolás (puffer)
- Megbízhatóság

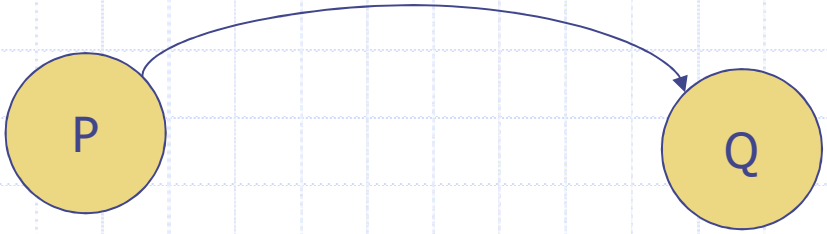
Folyamatok megnevezése (naming)

Az a módszer, amellyel a kommunikációban résztvevő folyamatok egymásra hivatkoznak.

- Közvetlen kommunikáció
- Közvetett kommunikáció
- Aszimmetrikus megnevezés
- Üzenetszórás

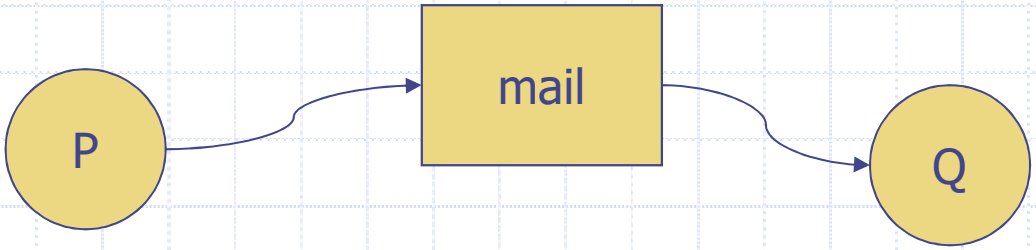
Közvetlen kommunikáció

- Folyamatok: *P*, *Q*
- üzenet küldés: *send*(*Q; message*),
- üzenet fogadás: message=*receive*(*P*)*;*
- csak egy csatorna létezik, és más folyamatok nem használhatják



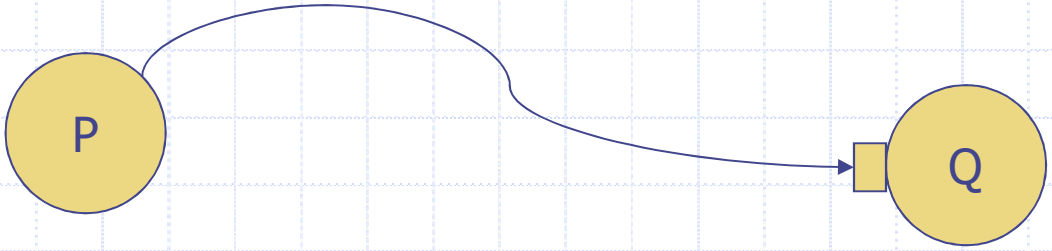
Közvetett kommunikáció

- Közbülső adatszerkezeten (pl. postaládán (mailbox)) keresztül valósul meg.
- Küldés: *send*(*mailbox, message*);
- Fogadás: message=*receive*(*mailbox*);
- Két folyamat között lehet több postaláda.
- Ugyanazt a postaládát több folyamat is használhatja egyidejűleg.
- A postaláda megosztása a vevők között:
  - csak két folyamat használhatja
  - egy időben csak egy vevő lehet
  - a rendszer választ, hogy melyik vevőnek küldi az üzenetet és erről értesíti az adót.



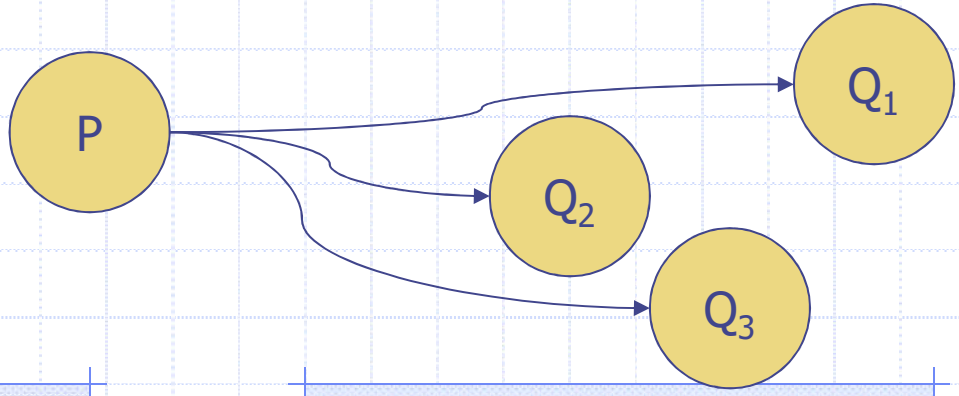
Aszimmetrikus megnevezés

- Adó vagy vevő megnevezi, hogy melyik folyamattal akar kommunikálni
- A másik fél egy kaput (port) használ, ezen keresztül több folyamathoz, is kapcsolódhat.
- Tipikus eset: a vevőhöz tartozik a kapu, az adóknak kell a vevő folyamatot és annak a kapuját megnevezni.
  - (Pl. szerver, szolgáltató folyamat)
- Küldés: *send*(*Q:port*,*message*);
- Fogadás: message=*receive*();



Üzenetszórás

- A közeg több folyamatot köt össze.
- Az üzenet amelyet több folyamat, esetleg mindegyik veheti.
- Küldés: *send*(*message, to_all*);
- Fogadás: message=*receive*();



Implicit szinkronizáció

Információcsere miatt.

- Tárolás nélküli átvitel
  - Nincs tárolás, így az adás és vétel egy időben zajlik (randevú).
- Véges kapacitású tároló
  - Egy időben csak az egyik folyamat használhatja (kölcsönös kizárás)
  - Vevő várakozik, amíg legalább egy elem nincs elküldve;
  - Adó várakozik, ha a puffer megtelt.
  - A két folyamat utasításai között adott sorrendiségnek kell teljesülni (precedencia).
- "Végtelen" puffer
  - Az adó folyamat nem várakozhat (pl. külső folyamat, amelyre az OS-nek nincs hatása).
  - A valóságban természetesen a puffer véges, így megtelhet.
  - Ilyenkor információvesztés: új adat figyelmen kívül hagyása vagy régi adat felülírása a pufferben.

Átviteli hibák kezelése

- Elosztott rendszer hibája
  - Folyamat befejeződése: adó vagy vevő folyamat váratlanul befejeződik
- Csatorna hibája
  - Elveszett, torzult üzenetek

Elosztott rendszer hibája

Adó folyamat áll le
- A vevő hiába várakozik a következő üzenetre.
- Nem holtpont (csak egy folyamatot érint).
- Felismerése: timeout.
- Feloldása: az OS terminálhatja a vevőt vagy jelzi neki az adó megszűnését.

Vevő folyamat áll le
- Az adó nulla vagy véges kapacitású puffer esetén várakozni kényszerül, megoldás az előzőekhez hasonló
- Végtelen puffer esetén az adó az esetleges nyugták megszűnéséből veheti észre a vevő megszűnését.

Timeout:
- egy időkorlát túllépése (az az időtartam, amelyen belül valaminek biztosan történnie kell)
- nehéz megállapítani.

Csatorna hibája

Elveszett, torzult üzenetek

- Az OS felismeri és újraküldi
- Az OS felismeri, az adót értesíti, így az újraküldi
- Vevő ismeri fel, az adót értesíti, így az újraküldi
- Torzult üzenetek felismerése egyszerű (HW vagy SW)
  - CRC (Cyclic Redundancy Codes)
  - checksum (pl: összeg mod 256)
  - ECC (Error Correcting Codes).
- Elveszett üzenetek detektálása bonyolultabb eljárás, pl. időkorlátok figyelése.